

Guide to AI Agents, Memory & Tool Calling in n8n

Single Agent + Dynamically Call Other Agents

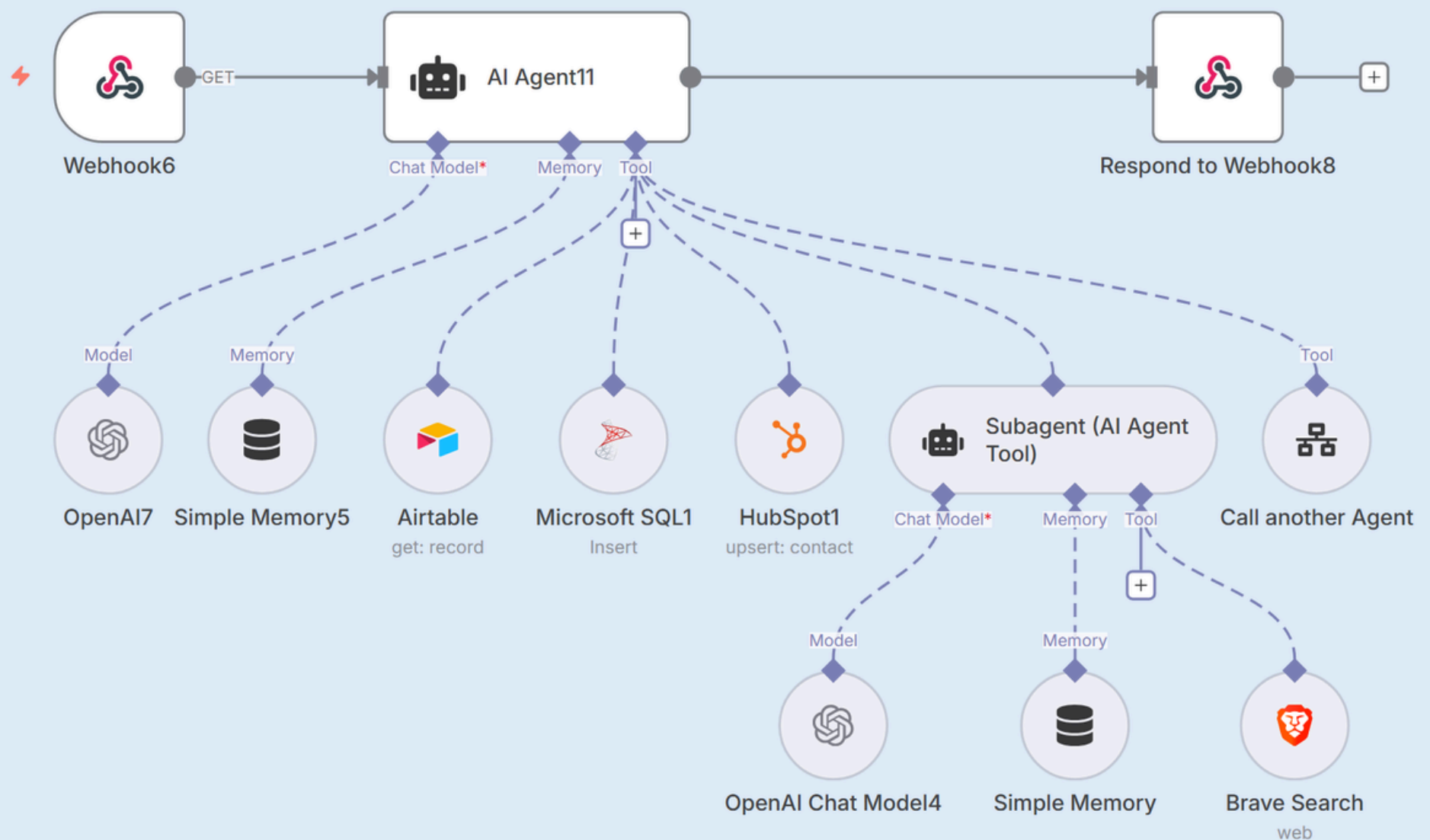


Image Source: Internet

1 • What Is an AI Agent?

An AI Agent is not just a language model. It is an LLM wrapped in a **reasoning loop** that can observe, think, act, and repeat. Where a plain LLM call takes input and returns output in one shot, an agent **decides which tools to call**, inspects the results, and keeps iterating until it has a final answer.



Plain LLM Call

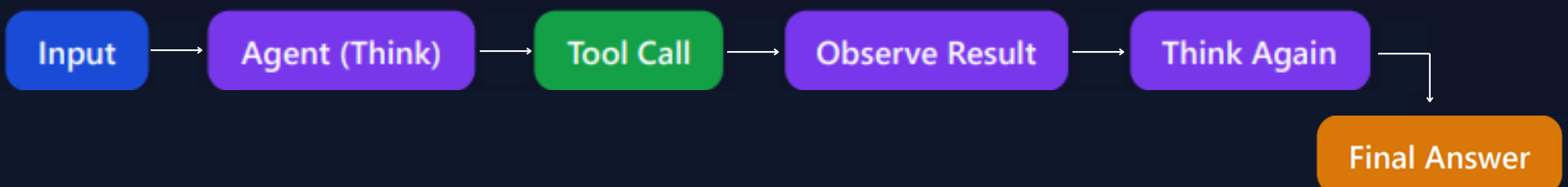
One-shot: prompt in → text out. No memory, no tools, no iteration. Like asking a question and getting a single answer.



AI Agent

Looping: prompt in → think → call tool → observe → think → call another tool → final answer. Autonomous decision-making.

THE AGENT REASONING LOOP



Key Insight

The agent may loop 1 time or 10 times depending on the complexity of the task. It decides autonomously when it has enough information to respond. This is what makes agents fundamentally different from chains — chains follow a fixed sequence, agents choose their own path.

Agent vs. Chain — When to Use Which

Criteria	Chain (Sequential)	Agent (Autonomous)
Steps known in advance?	Yes — fixed pipeline	No — agent decides at runtime
Needs external tools?	Sometimes (hardcoded)	Yes — agent picks which tools to call
Handles ambiguity?	Poorly — follows preset path	Well — reasons through edge cases
Token cost	Lower (one LLM call)	Higher (multiple LLM calls per loop)
Best for	Summarize, classify, transform	Research, triage, multi-step tasks

2 • The n8n AI Agent Node — Anatomy

The AI Agent node in n8n is a single node that orchestrates everything: the LLM, memory, tools, and output parsing. It has **4 sub-node slots** that you connect beneath it.



LLM Sub-node

Required. The language model that powers reasoning. Typically GPT-4o, Claude, or Gemini. This is the agent's "brain".



Memory Sub-node

Optional. Stores conversation history so the agent remembers prior turns. Window Buffer or Summary Memory.



Tool Sub-nodes

Optional. External capabilities: HTTP Request Tool, Code Tool, Workflow Tool, Calculator, etc. The agent decides when to call each.



Output Parser

Optional. Forces the agent's final answer into a specific JSON schema. Essential for downstream automation.

3 • Memory Types — Window Buffer vs Summary

Without memory, every message to the agent is a fresh conversation. The agent won't remember what the user asked 10 seconds ago. Memory nodes solve this by injecting prior conversation turns into the LLM's context window.

Window Buffer Memory

Stores the last **N messages** verbatim. Simple and fast. When the window is full, the oldest messages are dropped.



WINDOW BUFFER — HOW IT WORKS

```
// Window size = 4 messages
```

```
Turn 1: User: "What's the refund policy?"
```

```
Turn 2: Agent: "Refunds within 30 days ... "
```

```
Turn 3: User: "Does that apply to digital?"
```

```
Turn 4: Agent: "Yes, digital products too ... "
```

```
Turn 5: User: "And physical products?"
```

```
// Turn 1 is now dropped from memory
```

4 • Tool Calling — HTTP, Code & Workflow Tools

Tools are what give an agent its power. Without tools, an agent can only reason about its training data. With tools, it can **call APIs, run code, query databases, and trigger other workflows** — all autonomously.

How Tool Calling Works

AGENT TOOL-CALLING FLOW

- 1. Agent reads the user query
- 2. Agent decides: *"I need to look up the customer in Stripe"*
- 3. Agent calls **HTTP Request Tool** with Stripe API URL
- 4. Tool returns the customer record
- 5. Agent reads the result, decides it has enough info
- 6. Agent produces the final answer

Available Tool Types in n8n

Tool	What It Does	When to Use
HTTP Request Tool	Calls any REST API. You configure the URL template and the agent fills in parameters.	CRM lookups, payment APIs, external data fetches
Code Tool	Runs JavaScript/Python code. Agent provides input, code returns output.	Calculations, data transformations, regex parsing
Workflow Tool	Triggers a separate n8n workflow and returns its output to the agent.	Complex multi-step operations, database writes, sending emails
Calculator	Evaluates mathematical expressions.	Financial calculations, unit conversions
Vector Store Tool	Searches a vector database for relevant documents.	Knowledge base lookups, RAG-style retrieval

5 • Structured Outputs & Output Parsers

By default, an agent returns free-form text. That's fine for chat — but useless for automation. If the next node in your workflow expects `{ priority: "P0", category: "billing" }`, you need **guaranteed JSON output**. That's what Structured Output Parsers do.

The Problem Without Structured Outputs

UNSTRUCTURED AGENT OUTPUT

```
// Agent returns free-form text:
"This ticket is about a billing issue. I'd classify
it as high priority. The customer seems frustrated."

// Your downstream Switch node expects:
// {{ $json.priority }} → undefined ❌
// {{ $json.category }} → undefined ❌
```

The Solution: Structured Output Parser

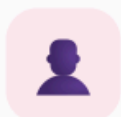
JSON SCHEMA FOR OUTPUT PARSER

```
{
  "type": "object",
  "properties": {
    "priority": {
      "type": "string",
      "enum": ["P0", "P1", "P2", "P3"]
    },
    "category": {
      "type": "string",
      "enum": ["billing", "outage", "feature", "how_to"]
    },
    "summary": { "type": "string" },
    "suggested_reply": { "type": "string" }
  },
  "required": ["priority", "category", "summary"]
}
```

6 • System Prompt Design for Agents

The system prompt is the most important configuration in your agent. It defines **who the agent is, what it can do, and what it must never do**. A well-crafted system prompt is the difference between a reliable production agent and a hallucinating mess.

The 5-Part System Prompt Framework



1. Identity & Role

"You are the Acme SaaS Support Agent. You triage incoming support tickets."



2. Task Description

"For each ticket: classify priority, identify category, look up customer in Stripe, draft a reply."



3. Available Tools

"You have access to: stripe_lookup, jira_create, knowledge_base_search."



4. Constraints

"Never promise refunds. Never share internal data. If unsure, escalate to a human."

7 • Agent Patterns & Architecture Decisions

Not every problem needs an agent. And not every agent needs every feature. Here are the most common patterns and when to use each.

Pattern 1: Single Agent, Multiple Tools

One agent with 3–5 tools. It reads the input, decides which tools to call, and produces a single structured output. Best for **ticket triage, lead qualification, customer support**.



Pattern 2: Chain → Agent Hybrid

Use a chain for the parts with predictable steps (e.g., summarize the input), then hand off to an agent for the parts that need autonomous decision-making. Saves tokens on the deterministic steps.

Pattern 3: Agent → Router → Specialist Agents

A "router" agent reads the input and decides which specialist agent should handle it. Each specialist has its own tools and prompt optimized for one task. Best for **complex workflows with very different task types**.



Start Simple

Always start with Pattern 1. Only move to 2 or 3 when you've proven that a single agent can't handle the complexity. Most production use cases work perfectly with one agent and 3–5 tools.

8 • Quick Reference Cheat Sheet

Concept	Key Details	n8n Node / Config
AI Agent Node	Orchestrates LLM + memory + tools in a reasoning loop	AI Agent node → attach sub-nodes below
Window Buffer Memory	Stores last N messages verbatim; fast, no extra LLM call	Window Buffer Memory sub-node; set window size 5–10
Summary Memory	Compresses older turns into a summary; good for long chats	Summary Buffer Memory sub-node; extra LLM call per turn
Session ID	Unique key per conversation to isolate memory	Set in Memory node config; use chat ID, user email, or UUID
HTTP Request Tool	Calls REST APIs; agent fills in parameters	HTTP Request Tool sub-node; set URL + auth
Code Tool	Runs JS/Python; agent provides input	Code Tool sub-node; write the function body
Workflow Tool	Triggers another n8n workflow	Workflow Tool sub-node; select target workflow
Structured Output Parser	Forces JSON schema on agent output; retries on failure	Structured Output Parser sub-node; define JSON schema
Max Iterations	Limits the think → act loop count	Agent node config; set 5–10 for production
System Prompt	Defines role, task, tools, constraints, output format	Agent node → System Message field